

Universidad Rafael Landívar.
Facultad de Ingeniería.
Ingeniería en Informática y Sistemas
Pensamiento Computacional, Sección 15.
Docente: Ing. Cindy García

Laboratorio # 15

Estudiantes: Fuentes Mosquera, Ana Daniela - 1118426

Guatemala, 17 de mayo de 2026.

Laboratorio # 15

Ejercicio 1: Tipos de Defectos - Seleccione la opción que mejor describa el tipo de defecto presentado.

- a. Un programa en C# para la recopilación de datos personales muestra el siguiente mensaje en pantalla:

“Ingresar su fecha de nacimiento y presionar Enter: ”

Como resultado, el sistema ha sido incapaz de interpretar las respuestas de los usuarios debido a que todas vienen en formatos distintos (ej. DD/MM/YYYY, MM-DDYYYY, YYYY-MM-DD, etc.)

- Defecto de Ambigüedad
 - Defecto de Inconsistencia
 - Defecto Lógico
 - Defecto matemático
- b. Se ha desarrollado un programa que calcula la suma, resta producto y cociente entre dos números ingresados por el usuario. Sin embargo, el programa presenta defectos cuando el usuario ingresa 0 en ambos valores pues no existe la división entre cero.
- Defecto de Escritura
 - Defecto matemático
 - Defecto de Ambigüedad
 - Defecto de Inconsistencia
- c. Se solicita desarrollar un programa que genere 20 números pares aleatorios entre 50 y 100. Deberá mostrar el listado de números en pantalla, resaltando en color cyan los números primos, y calcular las medidas de tendencia central.
- Defecto de Ambigüedad
 - Defecto de Escritura
 - Defecto matemático
 - Defecto de Inconsistencia

- d. Un sistema bancario en línea requiere que la contraseña de sus usuarios contenga entre 8 y 16 caracteres. A pesar de que se implementó una validación recientemente, se han detectado múltiples incidencias de fallo.

- Defecto de Ambigüedad
- Defecto de Escritura
- Defecto lógico
- Defecto de Inconsistencia

Ejercicio # 2: Pruebas de calidad (testing) – bottom up - Elabore un plan de pruebas para validar el correcto funcionamiento de los siguientes métodos. Para cada método deberá realizar tres pruebas, indicando:

- Descripción de la prueba
- Valores de entrada
- Resultado Esperado
- Resultado obtenido, si el método cumple o no la prueba.

El objetivo del ejercicio es evidenciar, si los hay, los defectos en cada método y sugerir una propuesta de mejora.

Propuesta: Añadir una validación que determine si las entradas se encuentran en el rango de 0 a 100 antes de realizar los cálculos.

- a. El método RetirarEfectivo() permite retirar dinero de una cuenta bancaria por medio de un cajero automático. Esta recibe dos parámetros; El saldo disponible en la cuenta y el monto que desea retirar.

Caso	Entrada	Resultado Esperado	Resultado Obtenido
Valores dentro del rango	Saldo = 80 Retiro = 20	Se ha retirado: 40Q Saldo actual: 40Q	Se realizó el retiro correctamente.
Monto mayor al saldo	Saldo = 40 Retiro = 70	Saldo insuficiente para realizar el retiro. Saldo actual: 40Q	No se pudo realizar ningún retiro.
Valor negativo	Saldo = -10 Retiro = 20	Error o Rechazo	No se pudo realizar ningún retiro ni ingreso.
Valor mayor a 100	Saldo = 120 Retiro = 20	Error o Rechazo	No se pudo realizar ningún retiro ni ingreso.

- b. La función `CalcularDescuento()` permite aplicar un descuento o rebaja al precio original de un producto. Para esto utiliza dos entradas; el precio original del producto y el porcentaje de descuento, entre 0 y 100, que debemos aplicar.

Caso	Entrada	Resultado Esperado	Resultado Obtenido
Valores dentro del rango	Precio = 100 Descuento = 20	Precio final: 80	Se realizo el descuento correctamente.
Descuento igual a 0	Precio = 100 Descuento = 0	Precio final: 100	No se pudo realizar ningún descuento.
Descuento negativo	Precio = 100 Descuento = -10	Error o Rechazo	No se pudo realizar ningún descuento.
Descuento mayor a 100	Precio = 100 Descuento = 120	Error o Rechazo	No se pudo realizar ningún descuento.

- c. El método `Depositar()` permite aumentar el saldo de una cuenta bancaria. Este recibe como entrada el saldo disponible del cliente y el monto que desea depositar.

Caso	Entrada	Resultado Esperado	Resultado Obtenido
Valores dentro del rango	Saldo = 50 Deposito = 30	Saldo final: 80	Se realizo el depósito correctamente.
Depósito igual a 0	Saldo = 50 Deposito = 0	Saldo final: 50	No se pudo realizar ningún deposito.
Depósito negativo	Saldo = 50 Deposito = -20	Error o Rechazo	No se pudo realizar ningún deposito.
Valor mayor a 100	Saldo = 150 Deposito = 20	Error o Rechazo	No se pudo realizar ningún deposito ni ingreso.

Ejercicio # 3: Seguimiento paso a paso (Debugging) - Crear un proyecto de consola en C# e ingresar el código proporcionado para simular el pago de un crédito personal.

```
static void Main()
{
    double capital = 1000;
    double tasa = 0.05;
    double intereses = 0;
    double abonos = 0;

    for (int mes = 1; mes <= 12 && capital > 0; mes++)
    {
        // Cálculo de intereses del mes
        intereses = capital * tasa;

        // Abono realizado cada mes
        abonos = 100 + (mes * 10);

        // Actualización del capital
        capital = capital + intereses - abonos;
    }
}
```

Deberá utilizar herramientas de depuración (debugging) mediante el uso de breakpoints para observar cómo cambian las variables durante la ejecución del programa.

Instrucciones:

- a. Ejecute el programa en modo depuración (Debug).
- b. Coloque un breakpoint dentro del ciclo for, específicamente en la línea donde se actualiza el capital.
- c. Utilice las herramientas de depuración:
 - Step Over (F10)
 - Ventana de variables locales
- d. Observe cómo cambian las siguientes variables durante cada iteración:
 - capital
 - intereses
 - abonos
 - tasa
 - mes

e. Tome capturas de pantalla donde se visualicen claramente los valores de las variables al finalizar:

- Mes 2

The screenshot shows the Visual Studio Code interface with a C# program being debugged. The code is for a savings account simulation. The debugger is paused at line 196, where the month 'mes' is 2. The 'Variables Locales' window shows the current values of variables: saldo (10), montoRetirado (0), precio (10), porcentaje (0), resultado (10), dinero (10), monto (0), capital (867), tasa (0.05), intereses (47), abono (120), and mes (2). The 'Pilas paralelas' window shows the call stack with 'Program.Main' at the top. The 'Pila de llamadas' window shows the current method call.

```
// Ejercicio 3: Seguimiento paso a paso (Debugging)

double capital = 1000;
double tasa = 0.05;
double intereses = 0;
double abono = 0;

for (int mes = 1; mes <= 12 && capital > 0; mes++)
{
    // Cálculo de intereses del mes
    intereses = capital * tasa;

    // Abono realizado cada mes
    abono = 100 + (mes * 10);

    // Actualización del capital
    capital += intereses - abono;
}

// Método a: RetirarEfectivo
static double RetirarEfectivo(double saldo, double montoRetirado)
{
    if (montoRetirado <= saldo)
```

Nombre	Valor	Tipo
saldo	10	double
montoRetirado	0	double
precio	10	double
porcentaje	0	double
resultado	10	double
dinero	10	double
monto	0	double
capital	867	double
tasa	0.05	double
intereses	47	double
abono	120	double
mes	2	int

Variables locales

Buscar (Ctrl+E)

Profundidad de búsqueda: 3

Nombre

Valor

Tipo

saldo 10 double

montoRetirado 0 double

precio 10 double

porcentaje 0 double

resultado 10 double

dinero 10 double

monto 0 double

capital 867 double

tasa 0.05 double

intereses 47 double

abono 120 double

mes 2 int

Pilas paralelas

Buscar (Ctrl+E)

Vista: Subproc

Filtro

* [8300] Subproceso principal

Ejecutando la función principal de la aplicación

Program.Main

Pila de llamadas

Buscar (Ctrl+E)

Ver todos los subprocesos

Mostrar código externo

Nombre

Leng

La15-A1-A3-AnaFuentes.exeLa15_A1_A3_AnaFuentes.Program.Main() Línea 196 C#

Automático Variables locales Inspección 1

Pila de llamadas Puntos de interr... Configuración d... Ventana Coman... Ventana Inmedi... Salida

Listo

8 / 37

- Mes 4

The screenshot shows the Visual Studio Code interface with a C# program being debugged. The code is for 'Ejercicio 3: Seguimiento paso a paso (Debugging)' and is paused at line 196. The 'Variables locales' window shows the state of variables like capital, tasa, intereses, abono, and mes. The 'Pila de llamadas' window shows the call stack with 'Main()' at the top.

Program.cs

```
177 // Ejercicio 3: Seguimiento paso a paso (Debugging)
178
179 double capital = 1000;
180 double tasa = 0.05;
181 double intereses = 0;
182 double abono = 0;
183
184 for (int mes = 1; mes <= 12 && capital > 0; mes++)
185 {
186     // Cálculo de intereses del mes
187     intereses = capital * tasa;
188
189     // Abono realizado cada mes
190     abono = 100 + (mes * 10);
191
192     // Actualización del capital
193     capital += intereses - abono;
194 }
195
196 // Método a: RetirarEfectivo
197 static double RetirarEfectivo(double saldo, double montoRetirado)
198 {
199     if (montoRetirado <= saldo)
200     {
201         // Retirar efectivo
202         saldo -= montoRetirado;
203     }
204     return saldo;
205 }
```

Variables locales

Nombre	Valor	Tipo
saldo	10	double
montoRetirado	0	double
precio	10	double
porcentaje	0	double
resultado	10	double
dinero	10	double
monito	0	double
capital	679.36750000000006	double
tasa	0.05	double
intereses	39.017500000000005	double
abono	140	double
mes	4	int

Pila de llamadas

Nombre	Leng
La15-A1-A3-AnaFuentes.exe!La15_A1_A3_AnaFuentes.Program.Main() Línea 196	C#

- Mes 6

Archivo Editar Ver Git Proyecto Compilar Depurar Prueba Herramientas Extensiones Ventana Ayuda Q Buscar La15-A1-A3-AnaFuentes

Proceso: [12964] La15-A1-A3-AnaFuentes Eventos del ciclo de vida Subproceso: [8300] Subproceso principal Marco de pila: La15_A1_A3-AnaFuentes.Program.Main

Program.cs

```
177 // Ejercicio 3: Seguimiento paso a paso (Debugging)
178
179 double capital = 1000;
180 double tasa = 0.05;
181 double intereses = 0;
182 double abono = 0;
183
184 for (int mes = 1; mes <= 12 && capital > 0; mes++) mes: 6
185 {
186     // Cálculo de intereses del mes
187     intereses = capital * tasa;
188
189     // Abono realizado cada mes
190     abono = 100 + (mes * 10);
191
192     // Actualización del capital
193     capital += intereses - abono;
194 }
195
196 // Método a: RetirarEfectivo
197 1 referencia
198 static double RetirarEfectivo(double saldo, double montoRetirado)
199 {
200     if (montoRetirado <= saldo)
201     {
```

100 % No se encontraron problemas. Línea: 196, Carácter: 13 SPC CRLF UTF-8 with BOM

Variables locales

Buscar (Ctrl+E) Profundidad de búsqueda: 3

Nombre	Valor	Tipo
saldo	10	double
montoRetirado	0	double
precio	10	double
porcentaje	0	double
resultado	10	double
dinero	10	double
monto	0	double
capital	431.50266875000011	double
tasa	0.05	double
intereses	28.166793750000007	double
abono	160	double
mes	6	int

Automático Variables locales Inspección 1

Pila de llamadas

Buscar (Ctrl+E) Ver todos los subprocesos Mostrar código externo

Nombre	Leng
La15-A1-A3-AnaFuentes.exelLa15_A1_A3-AnaFuentes.Program.Main()	C#

Pilas paralelas Herramientas de diagnóstico

8 / 37 37

- Mes 8

The screenshot shows the Visual Studio Code interface with a C# program being debugged. The code is for 'Ejercicio 3: Seguimiento paso a paso (Debugging)' and is paused at line 196. The 'Variables locales' window shows the current state of variables, and the 'Pila de llamadas' window shows the call stack.

Code Snippet:

```
177 // Ejercicio 3: Seguimiento paso a paso (Debugging)
178
179 double capital = 1000;
180 double tasa = 0.05;
181 double intereses = 0;
182 double abono = 0;
183
184 for (int mes = 1; mes <= 12 && capital > 0; mes++)
185 {
186     // Cálculo de intereses del mes
187     intereses = capital * tasa;
188
189     // Abono realizado cada mes
190     abono = 100 + (mes * 10);
191
192     // Actualización del capital
193     capital += intereses - abono;
194 }
195
196 // Método a: RetirarEfectivo
197 static double RetirarEfectivo(double saldo, double montoRetirado)
198 {
199     if (montoRetirado <= saldo)
200     {
201         // ...
202     }
203 }
```

Variables locales:

Nombre	Valor	Tipo
saldo	10	double
montoRetirado	0	double
precio	10	double
porcentaje	0	double
resultado	10	double
dinero	10	double
monto	0	double
capital	117.23169229687508	double
tasa	0.05	double
intereses	14.153890109375006	double
abono	180	double
mes	8	int

Pila de llamadas:

Nombre	Leng
La15-A1-A3-AnaFuentes.exelLa15_A1_A3_Anafuentes.Program.Main() Línea 196	C#